

→ I/O Port Programming (Code Only)

P0 P1 P2 P3
/ / / /
8 8 8 8 = 32 bits

③ Special port
read/write/timer
(RX) (TX)

① Lower bits of address (8 bit) (A0-A7)
② Upper bits of address (8 bit) (A8-A15)
16 bits

ALE = Address Latch Enable
= 0/1

EA = External Access

20 = GND

40 = VCC

Port I/O

- Port 0
- ① First 0 निल port becomes output
 - ② First 1 निल port becomes input
 - ③ Needs to use external pull-up register 10K

Sharan | ②

output example

```
BACK: MOV A, #55H
      MOV P0, A
      ACALL DELAY
      MOV A, #0AAH
      MOV P0, A
      ACALL DELAY → { DELAY code from }
      SJMP BACK   { prev chap }
```

input example

```
MOV A, #0FFH → FFH
MOV P0, A → { input ready? }
MOV A, P0
MOV P1, A
SJMP BACK
```

{ P1 → external pull-up resistor (दिया था) }

page-136


```

ORG 0
MOV A, #55
MOV P0, A
MOV P1, A
MOV P2, A
ACALL QSDelay
MOV A, #0AAH
MOV P0, A
MOV P1, A
MOV P2, A
ACALL QSDelay
SJMP BACK
    
```

QSDelay:

```

H3      MOV R5, #11
H2      MOV R4, #248
H1      MOV R3, #255
        DJNZ R3, H1
        DJNZ R4, H2
        DJNZ R5, H3
        RET
        END
    
```

4Mc

90ns

$\{x \times y \times z \times 4 \times 90\}$

target = 250

page 143

```

BACK: MOV P1, #55
      ACALL DELAY
      MOV P1, #0AAH
      ACALL DELAY
      SJMP BACK.
    
```

Sometimes we need to access only 1 or 2 bits of the port,

```

BACK: CPL P1.2] — pin 0, 1, 2
      ACALL DELAY
      SJMP BACK
    
```

```

AGAIN: SETB P1.2
        ACALL DELAY
        CLR P1.2
        ACALL DELAY
        SJMP AGAIN
    
```

Page 145

Create a sqw wave of 50% duty cycle on bit 0 of port 1

```

HERE: CPL P1.0
      LCALL DELAY
      SJMP HERE
    
```

SETB = set the bit
CLR, CPL, (JB), (JNB) → widely used

Example 4.3

```

SETB P1.2
MOV A, #45H
AGAIN JNB P1.2, AGAIN
MOV P0, A
SETB P2.3
CLR P2.3

```

Example 4.4

```

HERE JNB P2.3, HERE
SETB P1.5
CLR P1.5
SJMP HERE

```

Example 4.5

```

SETB P1.7
AGAIN: JB P1.2, OVER
MOV P2, #'N'
SJMP AGAIN
OVER: MOV P2, #'Y'
SJMP AGAIN

```

Example 4.6

```

SETB P1.7
AGAIN: MOV C, P1.2
JC OVER
MOV P2, #'N'
SJMP AGAIN
OVER: MOV P2, #'Y'
SJMP AGAIN

```

Shayan | 6

page 152

SETB PL.7

...

4 chapters done ☒

Arithmetic & Logic Instruction

+, -, .X, /

AND, OR, XOR
↓ ↓ ↓
ANL ORL XOR

ADD A, source; A += source

always 'A'

page 197

A → low bytes R7 → high bytes

40 = 7D

41 = ED

42 = C5

43 = 5B

44 = 30

MOV R0, #40H

MOV R2, #5

CLR A

MOV R7, A

AGAIN: ADD A, @R0

JNC NEXT

INC R7

INC R0

NEXT

DJNZ R2, AGAIN

Arithmetic

ADDC → 'Add Carry'
Adds 16 bits

ADD
Adds 8 bits

Math page 198

```
CLR C           ; make CY=0
MOV A, #0E7H
ADD A, #8DH
MOV R6, A
MOV A, #3CH
ADDC A, #3BH
MOV R7, A
```

```

      1
    3C E7
+   3B 8D
-----
    78 74

```

R7 R6

BCD

Unpacked → 2 lower bits a value (numbers 9 and 5)

Packed → upper 4 low both bits a value

→ ADD BCD

→ Unpacked ⇌ Packed

#ADD BCD

MOV A, #17H

ADD A, #28H

#3FH is not BCD

So, ADD '6' for correction

Hex → 16

Dec → 10

6(?)

Adjust BCD Addition

ADD A, B
DA A

{ Nibble = 4 bits }

Shayan | ②

Summary of DA

- ① If lower nibble > 9 or $AC=1$
add 0110 to lower 4 bits.
- ② If upper nibble > 9 or $CY=1$
add 0110 to upper 4 bits.

Page 203 Math
↪ Page 197

SUB - Subtraction

SUBB - subtraction with borrow

SUBB A

$$A = A - \text{source} - CY$$

SUBB

↳ when $CY=0$

- ① Take 2's complement of source operand
- ② Add minuend (A)
- ③ Invert the carry ($CY=1$)

```
CLR C
MOV  A, #4C
SUBB A, #6E
JNC  NEXT
CPL  A
INC  A
```

```
NEXT: MOV R1, A
```


SUBB

↳ When CY=1

Math page 206

Multiplication

MUL A, B

B → High byte

A → Low byte

Division

DIV A, B

A = Quotient

B = Remainder

Math 209 p

⑨ MOV A, #0FFH

MOV P1, A

MOV A, P1

MOV B, #10H

DIV AB

MOV R7, B

MOV B, #10H

DIV AB

MOV R6, B

MOV R5, A

⑩

FD10A	Q	R
	19	3
1910A	2	5
		2

Same code with loop

Shayan | (4)

(236)

	Q	R
EC / OA =	23	6
17 / OA =	2	3
		2

176H

logic MSB/LSB

Sign bit 0/1
+ -

How 8051 would represent -34H